



Programación Funcional

Clases teóricas

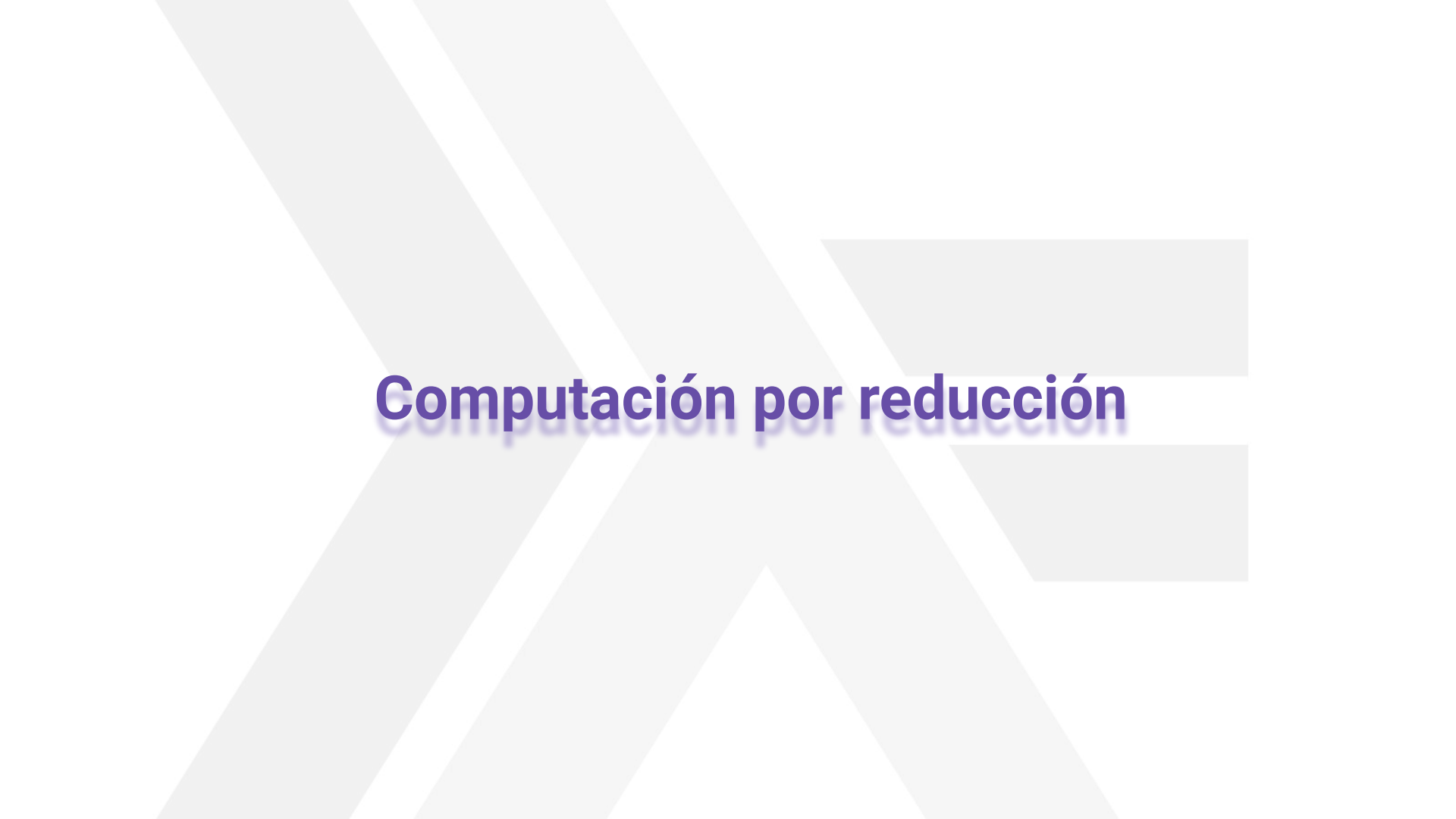
por Pablo E. “Fidel” Martínez López

4. Reducción

“Para transformar esta piedra en una gema tienes que ponerle otro nombre verdadero.”

Un mago de Terramar
Úrsula K. Le Guin





Computación por reducción

Computación por reducción

- ❑ ¿Cómo calcular mecánicamente el valor de una expresión?
 1. Repetir los pasos 2 y 3 hasta que no haya ninguna subexpresión que cumpla la condición dada por 2
 2. Encontrar una subexpresión que coincida con una instancia del lado izquierdo de una ecuación
 3. Reemplazarla por la correspondiente instancia del lado derecho
- ❑ Este proceso es llamado ***mecanismo de reducción***
 - ❑ O simplemente, **reducción**
 - ❑ Lo expresamos con →

Computación por reducción

- Algunas definiciones

- **Redex** (por *reducible expression*)

- Subexpresión que coincide con una instancia del lado izquierdo de una ecuación

- **Forma normal**

- Expresión que no contiene redexes

- La *reducción* reduce la distancia a la forma normal

Computación por reducción

❑ Mecanismo de reducción

Repetir hasta forma normal

Localizar un redex

Reemplazarlo

- ❑ Ya vimos ejemplos de reducciones en la clase 1
- ❑ ¿Qué propiedades tiene este mecanismo?

Propiedades de la reducción

- ❑ Propiedades

- ❑ **Confluencia**

- ¿la forma normal es única?

- ❑ **Normalización**

- ¿la forma normal siempre existe?

- ❑ **Orden de reducción**

- ¿importa en qué orden se eligen los redexes?



Confluencia

Confluencia

- ❑ **Confluencia** - ¿la forma normal es única?
 - ❑ ¿Qué implicaría si no fuese así?
 - ❑ ¡Habría más de un significado!
 - ❑ Es importante que lo sea para el significado
 - ❑ La forma de las definiciones permitidas influyen sobre la confluencia
 - ❑ En Haskell hay confluencia
 - ❑ No elaboraremos más sobre este punto



Normalización

Normalización

- ❑ Considerar las siguientes definiciones

```
inf :: ...
```

```
inf = inf + 1
```

```
detect :: ...
```

```
detect 42 = True
```

- ❑ ¿Tienen tipo?
- ❑ ¿Cómo reducen?

Normalización

- ❑ ¿Cuál es el tipo de `inf`? ¿Y cómo reduce?

```
inf :: ...
```

```
inf = inf + 1
```

Normalización

❑ ¿Cuál es el tipo de `inf`? ¿Y cómo reduce?

```
inf :: Int -- el resultado de la suma entre dos Ints
inf = inf + 1
```

Normalización

❑ ¿Cuál es el tipo de `inf`? ¿Y cómo reduce?

```
inf :: Int -- el resultado de la suma entre dos Ints  
inf = inf + 1
```

inf → ...

Normalización

❑ ¿Cuál es el tipo de `inf`? ¿Y cómo reduce?

```
inf :: Int -- el resultado de la suma entre dos Ints  
inf = inf + 1
```

`inf` → `inf+1` → ...

Normalización

❑ ¿Cuál es el tipo de `inf`? ¿Y cómo reduce?

```
inf :: Int -- el resultado de la suma entre dos Ints  
inf = inf + 1
```

`inf` $\rightarrow$ `inf+1` $\rightarrow$ `(inf+1)+1` $\rightarrow$...

Normalización

- ❑ ¿Cuál es el tipo de `inf`? ¿Y cómo reduce?

```
inf :: Int -- el resultado de la suma entre dos Ints
inf = inf + 1
```

`inf` $\rightarrow$ `inf+1` $\rightarrow$ `(inf+1)``+1` $\rightarrow$...

- ❑ ¡El redex nunca desaparece! La reducción no termina...
- ❑ ¿Es una ecuación orientada?

Normalización

❑ ¿Cuál es el tipo de **detect**? ¿Y cómo reduce?

```
detect :: ...
```

```
detect 42 = True
```

Normalización

❏ ¿Cuál es el tipo de **detect**? ¿Y cómo reduce?

```
detect ::      ->
```

```
detect 42 = True
```

Normalización

❏ ¿Cuál es el tipo de **detect**? ¿Y cómo reduce?

```
detect :: Int -> Bool
```

```
detect 42 = True
```

Normalización

❑ ¿Cuál es el tipo de **detect**? ¿Y cómo reduce?

```
detect :: Int -> Bool
```

```
detect 42 = True
```

```
detect 42 → ...
```

Normalización

❑ ¿Cuál es el tipo de `detect`? ¿Y cómo reduce?

```
detect :: Int -> Bool
```

```
detect 42 = True
```

```
detect 42 → True
```

Normalización

❑ ¿Cuál es el tipo de `detect`? ¿Y cómo reduce?

```
detect :: Int -> Bool
```

```
detect 42 = True
```

```
detect 42 → True
```

```
detect 21 →
```

Normalización

- ❑ ¿Cuál es el tipo de **detect**? ¿Y cómo reduce?

```
detect :: Int -> Bool
```

```
detect 42 = True
```

```
detect 42 → True
```

```
detect 21 ↯ -- ¡no hay ecuaciones!
```

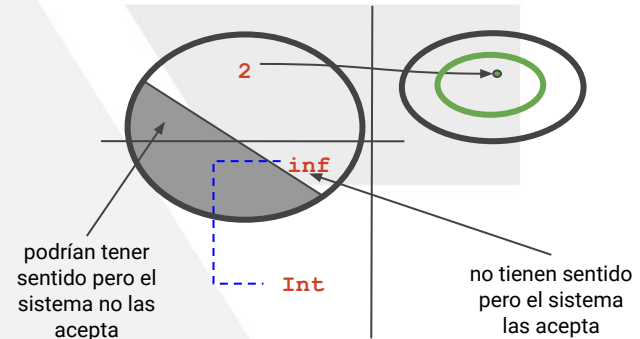
- ❑ ¡El redex no se puede reemplazar! Y no es forma normal...
 - ❑ ¿Es un redex?

Normalización

- ❑ **Normalización** - ¿La forma normal siempre existe?
 - ❑ ¡No!
 - ❑ ¿Cuáles son las formas de no llegar a forma normal?
 - ❑ Nunca termina de reducir (e.g. **inf**)
 - ❑ No puede seguir pero no llegó a forma normal (e.g. **detect**)
- ❑ Se llama *normalización* porque tiene que ver con la obtención de la forma *normal*
 - ❑ ¡No toda expresión tiene forma normal!

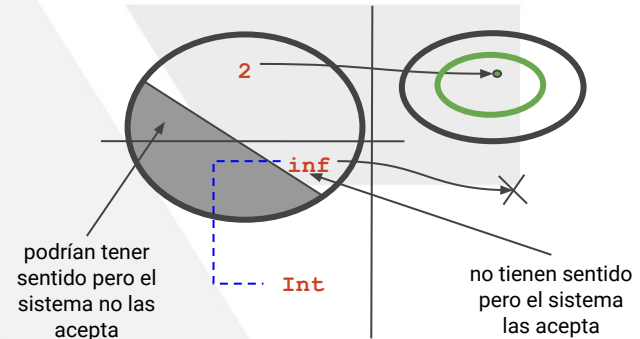
Normalización

- Consecuencias de la falta de formas normales
 - ¿Cuál es el valor de **inf**? ¿Existe?
 -



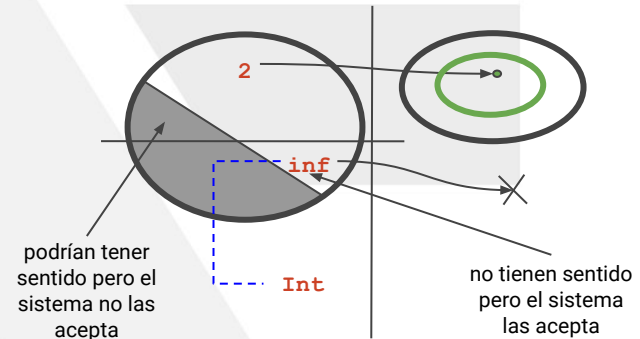
Normalización

- Consecuencias de la falta de formas normales
 - ¿Cuál es el valor de **inf**? ¿Existe?
 - Un número entero que sea igual a su sucesor



Normalización

- Consecuencias de la falta de formas normales
 - ¿Cuál es el valor de **inf**? ¿Existe?
 - Un número entero que sea igual a su sucesor
 - Pero **inf** tiene tipo...



Normalización

Consecuencias de la falta de formas normales

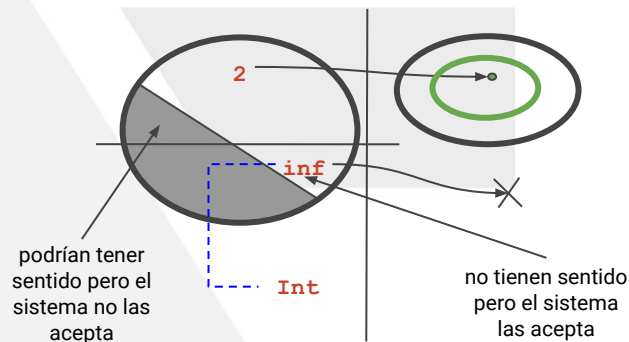
- ¿Cuál es el valor de **inf**? ¿Existe?

- Un número entero que sea igual a su sucesor

- Pero **inf** tiene tipo...

- Genera una discrepancia

- ¿Cómo la arreglamos?



Normalización

Consecuencias de la falta de formas normales

- ¿Cuál es el valor de **inf**? ¿Existe?

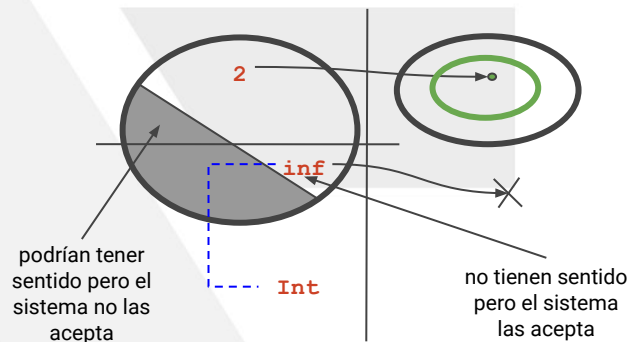
- Un número entero que sea igual a su sucesor

- Pero **inf** tiene tipo...

- Genera una discrepancia

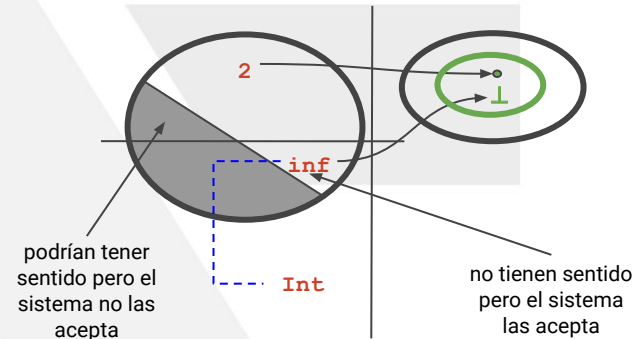
- ¿Cómo la arreglamos?

- Precisamos un
valor de error



Normalización

- Consecuencias de la falta de formas normales
 - Valor de error: BOTTOM ($\perp$)**

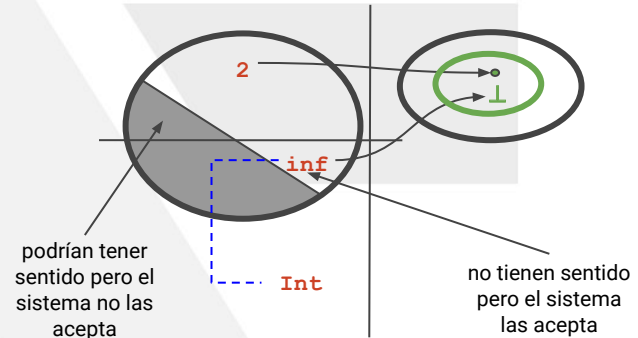


Normalización

- Consecuencias de la falta de formas normales

- Valor de error: BOTTOM ($\perp$)**

Valor teórico que representa a un error de cómputo



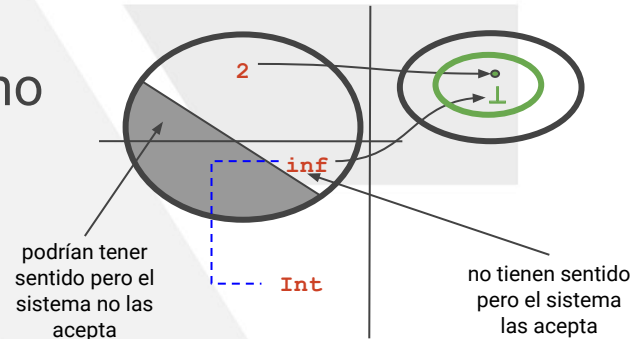
Normalización

Consecuencias de la falta de formas normales

Valor de error: BOTTOM ($\perp$)

Valor teórico que representa a un error de cómputo

- La reducción no termina
- La reducción no llega a destino



Normalización

Consecuencias de la falta de formas normales

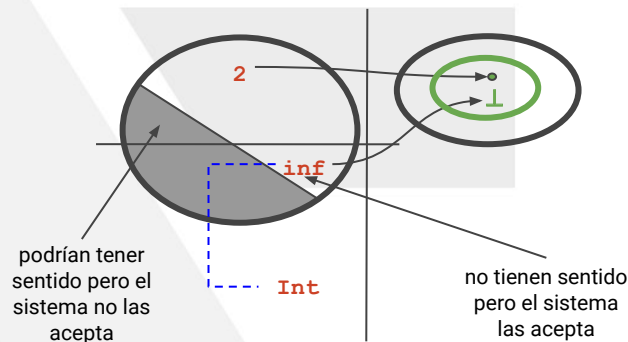
Valor de error: BOTTOM ($\perp$)

Valor teórico que representa a un error de cómputo

- La reducción no termina

- La reducción no llega

- ¡NO SE PUEDE manejar $\perp$ de manera operacional!



Normalización

- ❑ Bottom es un valor especial

Normalización

- ❑ Bottom es un valor especial
 - ❑ Usamos el símbolo $\perp$ para denotarlo

Normalización

- ❑ Bottom es un valor especial
 - ❑ Usamos el símbolo $\perp$ para denotarlo
 - ❑ Observar que es verde
 - ❑ No existe por sí mismo en el mundo de la expresiones

Normalización

- ❑ Bottom es un valor especial
 - ❑ Usamos el símbolo $\perp$ para denotarlo
 - ❑ Observar que es verde
 - ❑ No existe por sí mismo en el mundo de la expresiones
 - ❑ Es el valor de las expresiones que no terminan o no llegan a forma normal, un **valor de error**

Normalización

- ❑ Bottom es un valor especial
 - ❑ Usamos el símbolo $\perp$ para denotarlo
 - ❑ Observar que es verde
 - ❑ No existe por sí mismo en el mundo de las expresiones
 - ❑ Es el valor de las expresiones que no terminan o no llegan a forma normal, un **valor de error**
 - ❑ No se puede observar operacionalmente si una expresión denota $\perp$ (más de esto en un rato)

Normalización

❏ ¿Por qué se llama bottom?

Normalización

- ❑ ¿Por qué se llama bottom?
 - ❑ En inglés significa “el fondo”, “lo de más abajo”

Normalización

- ❑ ¿Por qué se llama bottom?
 - ❑ En inglés significa “el fondo”, “lo de más abajo”
 - ❑ Es el valor con menos significado,
“el de más abajo” en la escala de significados

Normalización

- ❑ ¿Por qué se llama bottom?
 - ❑ En inglés significa “el fondo”, “lo de más abajo”
 - ❑ Es el valor con menos significado, “el de más abajo” en la escala de significados
 - ❑ En slang también se usa para otra cosa que puede servir en castellano...

Normalización

- ❑ ¿Por qué se llama bottom?
 - ❑ En inglés significa “el fondo”, “lo de más abajo”
 - ❑ Es el valor con menos significado, “el de más abajo” en la escala de significados
 - ❑ En slang también se usa para otra cosa que puede servir en castellano...
 - ❑ El BOOM de Gobstones es bottom
 - ❑ Pero esto fue meramente accidental

Normalización

- Hay expresiones que no terminan de todos los tipos

```
inf = inf+1
```

```
bb = not bb
```

```
bc = if False then 'a' else bc
```

Normalización

- Hay expresiones que no terminan de todos los tipos

```
inf = inf+1
```

```
bb = not bb
```

```
bc = if False then 'a' else bc
```

- ¿A qué tipo pertenece, entonces, $\perp$?

Normalización

- Hay expresiones que no terminan de todos los tipos

```
inf = inf+1
```

```
bb = not bb
```

```
bc = if False then 'a' else bc
```

- ¿A qué tipo pertenece, entonces, $\perp$?

- TODOS los tipos contienen este valor

Normalización

- Hay expresiones que no terminan de todos los tipos

```
inf = inf+1
```

```
bb = not bb
```

```
bc = if False then 'a' else bc
```

- ¿A qué tipo pertenece, entonces, $\perp$?
- TODOS los tipos contienen este valor
- ¿Se puede definir una expresión polimórfica que dé $\perp$?

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que dé $\perp$?

`bottom :: ...`

`bottom = ...`

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

`bottom :: a`

`bottom = bottom`

- ❑ Para no terminación, se puede

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

`bottom :: a`

`bottom = bottom`

- ❑ Para no terminación, se puede
 - ❑ De hecho, $\perp$ es el único valor con tipo `a`

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

`bottom :: a`

`bottom = bottom`

- ❑ Para no terminación, se puede
 - ❑ De hecho, $\perp$ es el único valor con tipo `a`
 - ❑ En Haskell se llama `undefined` en lugar de `bottom`

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

`bottom :: a`

`bottom = bottom`

- ❑ Para no terminación, se puede
 - ❑ De hecho, $\perp$ es el único valor con tipo `a`
 - ❑ En Haskell se llama `undefined` en lugar de `bottom`
- ❑ ¿Y se puede definir una que no llegue a forma normal?
 - ❑ Debería no tener ecuaciones...

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

error :: ...

error = ...

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

error :: ...

error = ...

- ❑ Que no llegue a forma normal, no se puede

Normalización

- ❑ ¿Se puede definir una expresión polimórfica que de $\perp$?

```
error :: String -> a  
-- Predefinida en Haskell
```

- ❑ Que no llegue a forma normal, no se puede
 - ❑ Debe ser parte del lenguaje